

Payment Service Design Document

Purpose: Process payments, maintain payment history, manage refunds, and publish payment outcome events. Payment Service is the source of truth for all payment-related data.

Responsibilities

- Process payment requests
- Handle refunds
- Maintain payment history
- Publish payment events

Owned Entities

Payment, PaymentTransaction, Refund

Source of Truth

Payment status, payment transactions, provider references, refund history.

High-Level Architecture

Booking publishes PaymentRequested → Payment Service processes payment → Updates database → Writes Outbox (optional) → Publishes PaymentSucceeded / PaymentFailed → Booking Saga continues.

ER Diagram

```
Payment ----- payment_id (PK) booking_id amount
currency status provider_reference created_at updated_at version 1 | *
PaymentTransaction ----- transaction_id (PK)
payment_id (FK) provider transaction_type provider_reference status
amount created_at 1 | * Refund ----- refund_id (PK)
payment_id (FK) refund_amount status provider_reference created_at
```

Database Schema

Payment: payment_id, booking_id, amount, currency, status, provider_reference, created_at, updated_at, version

PaymentTransaction: transaction_id, payment_id, provider, transaction_type, provider_reference, status, amount, created_at

Refund: refund_id, payment_id, refund_amount, status, provider_reference, created_at

Indexes

Payment(booking_id UNIQUE); Payment(status); Payment(provider_reference);
PaymentTransaction(payment_id); PaymentTransaction(provider_reference);
Refund(payment_id)

Published Events

PaymentSucceeded, PaymentFailed, RefundCompleted

Consumed Events

PaymentRequested

Business Rules

Each booking can have one primary payment record. Every provider interaction is stored as a PaymentTransaction. Refunds reference the original payment. Duplicate PaymentRequested events must be handled idempotently.

Payment State Machine

PENDING → PROCESSING → SUCCESS or FAILED. SUCCESS → REFUNDED (partial/full refunds supported).

Failure Handling

Provider timeout → Retry with exponential backoff. Permanent failures publish PaymentFailed. Refund failures are retried and monitored.

Reliability

Use Transactional Outbox for publishing events, idempotent Kafka consumers, optimistic locking on Payment, metrics, tracing and audit logging.